

Universidad — Machine Learning

Shell Sort y Algoritmos Genéticos

Teoría, Implementación y Optimización
de Secuencias de Brechas

Práctica 5 — Curso de Machine Learning

Pedro Alonso

Junio 2026

1. Resumen

Este documento presenta una investigación experimental en torno al algoritmo de ordenamiento Shellsort y su optimización paramétrica a través de Algoritmos Genéticos (AG). Shellsort, propuesto por Donald Shell en 1959, generaliza el ordenamiento por inserción mediante una secuencia decreciente de brechas (*gaps*). Su complejidad temporal varía de $O(n^2)$ a $O(n \log^2 n)$ dependiendo exclusivamente de la secuencia de brechas elegida, un problema que ha ocupado a los investigadores por más de seis décadas. Se presentan y comparan las secuencias clásicas (Shell 1959, Hibbard 1963, Knuth 1973, Sedgewick 1986, Ciura 2001) con sus complejidades teóricas asociadas.

Los Algoritmos Genéticos proporcionan un marco de optimización no derivativo capaz de explorar vastos espacios combinatorios. Se expone su base matemática (Teorema del Esquema) y sus componentes principales. Posteriormente, se propone una implementación práctica que emplea un AG generacional clásico para descubrir secuencias de brechas competitivas en el escenario del peor caso (arreglo estrictamente inverso), validando de forma empírica y reproducible el poder de estas técnicas evolutivas.

Palabras clave: Shellsort, secuencias de brechas, algoritmos genéticos, optimización combinatoria, ordenamiento por inserción, complejidad computacional.

Tabla de Contenidos

1. Resumen	3
2. Introducción	6
2.1. Objetivo del reporte	6
2.2. Organización	6
3. Ordenamiento por Inserción: el fundamento	7
3.1. Idea central	7
3.2. Pseudocódigo	7
3.3. Análisis de complejidad	7
4. Shellsort: Ordenamiento por Incrementos	8
4.1. Motivación: mover elementos lejos en un paso	8
4.2. Definición formal	8
4.3. Pseudocódigo	8
4.4. Propiedad fundamental: el h-orden se preserva	9
4.5. Implementación en la práctica	9
5. Secuencias de Brechas: Seis Décadas de Investigación	10
5.1. ¿Qué hace buena a una secuencia?	10
5.2. Shell (1959): la secuencia original	10
5.3. Hibbard (1963): la primera secuencia coprima	10
5.4. Knuth (1973): la secuencia práctica estándar	10
5.5. Sedgewick (1986): el mejor peor caso conocido	11
5.6. Ciura (2001): la mejor secuencia empírica	11
5.7. Tabla comparativa	11
5.8. Investigación reciente: optimización paramétrica	12
6. Algoritmos Genéticos: Teoría y Fundamentos	13
6.1. ¿Qué es un Algoritmo Genético?	13
6.2. Fundamento matemático: el Teorema del Esquema	13
6.3. Componentes del AG canónico	14
6.4. Algoritmo canónico completo	16
6.5. ¿Por qué un AG para encontrar gaps?	17
7. Diseño del AG para Optimización de Brechas	18
7.1. Arquitectura del código	18
7.2. Reproducibilidad: el módulo <code>datos.py</code>	18
7.3. Codificación del cromosoma	18
7.4. Función de aptitud: promedio sobre múltiples muestras	19
7.5. Selección por torneo	19
7.6. Cruza de un punto	20
7.7. Mutación por reemplazo	20
7.8. Bucle principal y elitismo	20
7.9. Parámetros del AG	21
8. Resultados Experimentales	22
8.1. Metodología de benchmark	22
8.2. Resultados comparativos	22
8.3. Análisis de los resultados	22
8.4. Curva de aprendizaje del AG	23

9. Conclusiones	24
10. Referencias	25
Bibliografía	26

2. Introducción

El ordenamiento de datos es uno de los problemas más estudiados en ciencias de la computación. Su importancia práctica es enorme: bases de datos, sistemas operativos, compiladores y aplicaciones científicas dependen de ordenar colecciones de datos de manera eficiente.

Entre los algoritmos de ordenamiento por comparación, el límite inferior teórico es $\Omega(n \log n)$, demostrado mediante el modelo del árbol de decisión [1]. Este límite es alcanzado por Mergesort y Heapsort en el peor caso, y por Quicksort en el caso promedio. Sin embargo, existe un algoritmo que ocupa un lugar peculiar en esta jerarquía: **Shellsort**.

Shellsort logra desempeños intermedios entre $O(n^2)$ y $O(n \log n)$ con una implementación notablemente simple — básicamente, un bucle anidado —, y su complejidad precisa depende de un parámetro que el usuario elige: la **secuencia de brechas**. Esta dependencia paramétrica convierte a Shellsort en un campo de estudio vivo: más de sesenta años después de su publicación, aún no se conoce la secuencia óptima.

La pregunta «¿cuál es la mejor secuencia de brechas?» es naturalmente un problema de optimización. Los enfoques históricos han sido analíticos (derivar fórmulas matemáticas que garanticen buenas propiedades) o empíricos (explorar combinaciones y medir). Esta práctica agrega un tercer enfoque: usar un **Algoritmo Genético** para descubrir computacionalmente secuencias competitivas.

2.1. Objetivo del reporte

El objetivo central de esta práctica es analizar experimentalmente el impacto de la secuencia de brechas en el rendimiento algorítmico de Shellsort, y demostrar la viabilidad de utilizar Algoritmos Genéticos como herramienta de optimización de parámetros combinatorios en diseño de algoritmos. Se evalúa el descubrimiento empírico de secuencias robustas frente a casos límite (el peor escenario posible).

2.2. Organización

El reporte se organiza como sigue:

1. **Sección 2:** ordenamiento por inserción, fundamento de Shellsort.
2. **Sección 3:** Shellsort, definición formal, pseudocódigo y propiedades.
3. **Sección 4:** secuencias de brechas clásicas con análisis de complejidad.
4. **Sección 5:** algoritmos genéticos, teoría y componentes.
5. **Sección 6:** diseño del AG para optimización de brechas y análisis del código.
6. **Sección 7:** resultados experimentales y comparación.
7. **Sección 8:** conclusiones y extensiones.

3. Ordenamiento por Inserción: el fundamento

Para comprender Shellsort es indispensable dominar el ordenamiento por inserción, del cual es una generalización directa.

3.1. Idea central

Imagine ordenar una mano de naipes recibidos uno a uno. Al recibir una carta nueva, se busca su posición correcta en las cartas ya sostenidas y se inserta allí, desplazando las cartas mayores hacia la derecha. El ordenamiento por inserción es exactamente este proceso aplicado a un arreglo.

Formalmente, para un arreglo $A[0..n-1]$:

- Invariante de bucle: al iniciar la iteración i , el subarreglo $A[0..i-1]$ está ordenado.
- En cada iteración: se toma $A[i]$ como **clave** y se inserta en su posición correcta dentro del subarreglo ordenado, desplazando los elementos mayores.

3.2. Pseudocódigo

```
INSERCIÓN(A, n):
  para i desde 1 hasta n-1:
    clave := A[i]
    j := i - 1
    mientras j >= 0 y A[j] > clave:
      A[j+1] := A[j]           // desplazamiento
      j := j - 1
    A[j+1] := clave           // colocación
```

3.3. Análisis de complejidad

Cada elemento $A[i]$ puede desplazarse como máximo i posiciones hacia la izquierda. Si definimos I como el número de **inversiones** del arreglo (pares (i, j) con $i < j$ y $A[i] > A[j]$), se puede demostrar que el número de operaciones es:

$$T(A) = n - 1 + I(A) \quad (1)$$

donde $n - 1$ son las comparaciones que terminan el bucle **while** (las «falsas») y $I(A)$ es el número exacto de desplazamientos y comparaciones verdaderas [2].

- **Mejor caso:** A ya ordenado, $I = 0$, costo $\Theta(n)$.
- **Peor caso:** A en orden inverso, $I = \binom{n}{2} = \frac{n(n-1)}{2}$, costo $\Theta(n^2)$.
- **Caso promedio:** si las permutaciones son equiprobables, $E[I] = \frac{\binom{n}{2}}{2}$, costo $\Theta(n^2)$.

3.3.1. La clave: los elementos se mueven de a una posición

El cuello de botella del ordenamiento por inserción es que cada desplazamiento mueve un elemento exactamente **una posición**. Un elemento que debe recorrer d posiciones hasta su destino requiere exactamente d desplazamientos. Esta restricción es lo que Shellsort rompe.

4. Shellsort: Ordenamiento por Incrementos

4.1. Motivación: mover elementos lejos en un paso

Sea un arreglo de $n = 8$ elementos (el peor caso de la inserción clásica): $[8, 7, 6, 5, 4, 3, 2, 1]$. El ordenamiento por inserción estándar requiere $\binom{8}{2} = 28$ operaciones. Shellsort busca romper esta limitación realizando intercambios a larga distancia.

Ejemplo paso a paso (Gaps = {4, 1}):

- **Pasada $h = 4$:** El arreglo se visualiza como 4 subarreglos intercalados:
 Sub 0: $\{A[0]=8, A[4]=4\} \rightarrow$ al ordenarlo queda $\{4, 8\}$ Sub 1: $\{A[1]=7, A[5]=3\} \rightarrow$ al ordenarlo queda $\{3, 7\}$ Sub 2: $\{A[2]=6, A[6]=2\} \rightarrow$ al ordenarlo queda $\{2, 6\}$ Sub 3: $\{A[3]=5, A[7]=1\} \rightarrow$ al ordenarlo queda $\{1, 5\}$ El arreglo completo se transforma en $[4, 3, 2, 1, 8, 7, 6, 5]$. Se logran grandes saltos (ej: el 1 saltó desde la posición 7 hasta la 3 en un solo intercambio).
- **Pasada $h = 1$:** Se aplica inserción tradicional sobre el nuevo arreglo. Como ahora todos los elementos están a corta distancia de su destino final, el proceso toma muchas menos comparaciones. El 1 solo viaja de la posición 3 a la 0 (3 pasos, en vez de 7).

El propósito fundamental de las brechas es lograr un alto grado de «pre-ordenamiento» con costo reducido para que la inevitable pasada final con $h = 1$ actúe sobre un arreglo sumamente cercano a su forma ordenada.

4.2. Definición formal

Sea $\mathcal{G} = (h_1, h_2, \dots, h_t)$ una secuencia decreciente de enteros positivos con $h_t = 1$. Shellsort ejecuta t pasadas:

Pasada k : aplica el ordenamiento por inserción con paso h_k sobre el arreglo. Esto divide el arreglo en h_k subarreglos entrelazados:

$$S_r = \{A[r], A[r + h_k], A[r + 2h_k], \dots\} \quad \text{para } r = 0, 1, \dots, h_k - 1 \quad (2)$$

Cada S_r se ordena independientemente mediante inserción. Al terminar la pasada, el arreglo está **h_k -ordenado**: para todo i , $A[i] \leq A[i + h_k]$.

4.3. Pseudocódigo

```
SHELLSORT(A, n, G = (h_1, ..., h_t)):
  para k desde 1 hasta t:
    h := G[k]
    para i desde h hasta n-1:           // inserción con paso h
      clave := A[i]
      j := i
      mientras j >= h y A[j-h] > clave:
        A[j] := A[j-h]                 // desplazamiento en paso h
        j := j - h
      A[j] := clave
```

Nótese que el bucle interno es **idéntico** al del ordenamiento por inserción, con h reemplazando el paso 1.

4.4. Propiedad fundamental: el h -orden se preserva

Lema (Knuth 1973 [1]): Si un arreglo está p -ordenado y se le aplica una pasada de Shellsort con brecha q , el arreglo resultante está simultáneamente p -ordenado y q -ordenado.

Este lema es la razón de ser de Shellsort: el trabajo de cada pasada **no deshace** el de las anteriores, solo agrega más estructura. Las pasadas con brechas grandes establecen un orden «grueso» que las pasadas posteriores solo refinan. En consecuencia, cuando se llega a la pasada con $h = 1$, el arreglo tiene pocas inversiones y el ordenamiento por inserción termina en tiempo casi lineal.

4.5. Implementación en la práctica

La función `shell_sort` en `shell_sort.py` implementa el algoritmo con un **contador de costo determinístico**:

```
def shell_sort(arr, gaps):
    a = np.array(arr, copy=True)  # no muta el arreglo original
    n = len(a)
    costo = 0

    for gap in gaps:
        for i in range(gap, n):
            valor_actual = a[i]
            j = i
            while j >= gap and a[j - gap] > valor_actual:
                costo += 1  # comparación verdadera
                a[j] = a[j - gap]  # desplazamiento
                costo += 1  # costo del desplazamiento
                j -= gap
            if j >= gap:
                costo += 1  # comparación falsa que terminó el
while
        a[j] = valor_actual
    return a, costo
```

La métrica de costo es **determinística**: no depende de la velocidad del procesador ni de la carga del sistema. Esto la hace ideal como función objetivo para el AG, ya que dos ejecuciones del mismo código con las mismas entradas producen exactamente el mismo costo.

4.5.1. Decisión de diseño: ¿por qué contar comparaciones y movimientos?

El tiempo de reloj (`time.time()`) es ruidoso: varía entre ejecuciones por interrupciones del sistema operativo, caché del procesador y carga del sistema. En contraste, contar operaciones da un proxy exacto del trabajo real que el algoritmo realiza, comparable entre máquinas y reproducible.

5. Secuencias de Brechas: Seis Décadas de Investigación

La complejidad de Shellsort depende exclusivamente de la secuencia de brechas. Esta sección presenta las secuencias más importantes en orden cronológico, con sus propiedades matemáticas y complejidades.

5.1. ¿Qué hace buena a una secuencia?

Antes de revisar las secuencias específicas, establece cuatro propiedades deseables [3]:

1. **Terminar en 1:** indispensable. Sin la pasada con $h = 1$, no hay garantía de orden total.
2. **Ser decreciente:** Shellsort procesa brechas de mayor a menor (primero orden grueso, luego fino).
3. **Coprimalidad:** si $\gcd(h_i, h_j) > 1$ para dos brechas consecutivas, los subarreglos h_i y h_j comparten elementos y las pasadas se solapan, desperdiciando comparaciones. La secuencia de Shell original sufre este problema gravemente.
4. **Ritmo de decrecimiento balanceado:** si las brechas caen muy rápido, pocas pasadas dan bajo «pre-acomodo»; si caen muy lento, demasiadas pasadas cuestan fijo.

5.2. Shell (1959): la secuencia original

Donald Shell propuso dividir repetidamente por 2 [4]:

$$h_k = \lfloor n/2^k \rfloor, \quad k = 1, 2, \dots \quad (3)$$

Para $n = 2000$: (1000, 500, 250, 125, 62, 31, 15, 7, 3, 1).

Problema: todas las brechas son potencias de 2 multiplicadas por factores comunes. En particular, las brechas pares nunca mezclan los subarreglos pares e impares hasta la pasada final con $h = 1$. Formalmente, si n es potencia de 2, los dos subarreglos de índices pares e impares nunca interactúan hasta la última pasada, lo que la convierte esencialmente en dos problemas de tamaño $\frac{n}{2}$ seguidos de un paso de mezcla [1].

Complejidad: $\Theta(n^2)$ en el peor caso [5].

5.3. Hibbard (1963): la primera secuencia coprime

Thomas Hibbard observó que la mala interacción de la secuencia de Shell proviene de sus factores comunes [6]. Propuso:

$$h_k = 2^k - 1 = 1, 3, 7, 15, 31, 63, 127, 255, \dots \quad (4)$$

Para $n = 2000$: (1023, 511, 255, 127, 63, 31, 15, 7, 3, 1).

Propiedad clave: brechas consecutivas son coprimas. Demostración: $\gcd(2^k - 1, 2^{k+1} - 1) = \gcd(2^k - 1, 2^k + 1) = \gcd(2^k - 1, 2) = 1$ (ya que $2^k - 1$ es impar).

Complejidad: $O(n^{\frac{3}{2}})$ en el peor caso [6]. Es la primera secuencia con complejidad demostrada mejor que $O(n^2)$.

5.4. Knuth (1973): la secuencia práctica estándar

Donald Knuth popularizó la recurrencia [1]:

$$h_1 = 1, \quad h_{\{k+1\}} = 3h_k + 1 \quad (5)$$

Produciendo: 1, 4, 13, 40, 121, 364, 1093, ... En la implementación se usan las brechas menores que $\frac{n}{3}$. Para $n = 2000$: (1093, 364, 121, 40, 13, 4, 1).

Complejidad: $O(n^{3/2})$ en el peor caso. La ventaja sobre Hibbard es práctica: la secuencia $3h + 1$ tiene una razón de crecimiento ≈ 3 , que empíricamente da mejor rendimiento que el factor 2 de Hibbard.

Uso en la industria: es la secuencia implementada en la mayoría de los libros de texto y en implementaciones de referencia.

5.5. Sedgewick (1986): el mejor peor caso conocido

Robert Sedgewick propuso una familia de secuencias que mezclan dos fórmulas para maximizar la coprimalidad entre brechas no consecutivas [7]:

$$h_k = \begin{cases} 9(4^j - 2^j) + 1 & \text{si } k = 2j, \\ 4^j - 3 \cdot 2^j + 1 & \text{si } k = 2j - 1 \end{cases} \quad (6)$$

Produciendo: 1, 5, 19, 41, 109, 209, 505, 929, ... Para $n = 2000$: (1073, 281, 77, 23, 8, 1).

Complejidad: $O(n^{4/3})$ en el peor caso — la mejor cota demostrada analíticamente hasta 2025 [7].

5.6. Ciura (2001): la mejor secuencia empírica

Marcin Ciura adoptó un enfoque radicalmente distinto: en lugar de derivar una fórmula, buscó experimentalmente los incrementos que minimizaban el número de comparaciones en el caso promedio para arreglos de distintos tamaños [8]:

$$1, 4, 10, 23, 57, 132, 301, 701, \dots \quad (7)$$

Para $n = 2000$: (701, 301, 132, 57, 23, 10, 4, 1).

No se conoce una fórmula cerrada que genere esta secuencia; los términos más allá de 701 se calculan multiplicando por ≈ 2.25 . Esta secuencia ha sido el estándar de referencia durante más de dos décadas y es el antecedente conceptual más cercano al enfoque de esta práctica: si Ciura descubrió la mejor secuencia buscando empíricamente, ¿podría un Algoritmo Genético hacer lo mismo de forma automática?

5.7. Tabla comparativa

Secuencia	Año	Peor caso	Para n=2000
Shell	1959	$\Theta(n^2)$	1000, 500, 250, 125, 62, 31, 15, 7, 3, 1
Hibbard	1963	$O(n^{3/2})$	1023, 511, 255, 127, 63, 31, 15, 7, 3, 1
Knuth	1973	$O(n^{3/2})$	1093, 364, 121, 40, 13, 4, 1
Sedgewick	1986	$O(n^{4/3})$	1073, 281, 77, 23, 8, 1
Ciura	2001	Empírica	701, 301, 132, 57, 23, 10, 4, 1
Skean et al.	2023	$O(n^{4/3})?$	Paramétrica

Tabla 1: Secuencias de brechas clásicas. Todas terminan en 1.

5.8. Investigación reciente: optimización paramétrica

Skean, Ehrenborg y Jaromczyk (2023) [9] estudiaron secuencias de la forma $h_k = \lfloor a \cdot b^k \rfloor$ con parámetros (a, b) optimizados por búsqueda en grilla, encontrando secuencias competitivas con Ciura. Su trabajo confirma que el espacio de secuencias tiene estructura optimizable computacionalmente — exactamente la premisa de esta práctica.

6. Algoritmos Genéticos: Teoría y Fundamentos

6.1. ¿Qué es un Algoritmo Genético?

Un Algoritmo Genético (AG) es una técnica de búsqueda y optimización inspirada en la evolución biológica por selección natural, propuesta por John Holland en 1975 [10]. Pertenecce a la familia de los **algoritmos evolutivos** y opera manteniendo una **población** de soluciones candidatas que evoluciona generación a generación.

La analogía con la biología es explícita y deliberada:

Biología	Algoritmo Genético
Individuo	Solución candidata
Cromosoma / Genotipo	Representación codificada (vector)
Fenotipo	Solución decodificada en el dominio del problema
Aptitud (fitness)	Calidad numérica de la solución
Selección natural	Selección de los más aptos para reproducirse
Reproducción sexual	Cruza (crossover) entre dos padres
Mutación genética	Perturbación aleatoria de genes
Generación	Una iteración del bucle principal

Tabla 2: Analogía biológica del Algoritmo Genético.

La clave de los AG es la tensión entre **explotación** (refinar soluciones buenas conocidas) y **exploración** (descubrir nuevas regiones del espacio de búsqueda). La selección impulsa la explotación; la cruce y la mutación impulsan la exploración [11].

6.2. Fundamento matemático: el Teorema del Esquema

El éxito de los AG no es magia: tiene una explicación matemática formal desarrollada por Holland [10] y analizada en profundidad por Goldberg [11].

6.2.1. Definición de esquema

Un **esquema** (schema) H es una plantilla sobre el alfabeto $\{0, 1, *\}$ donde $*$ significa «cualquier valor». Por ejemplo, el esquema $H = 1 * 0 *$ describe todos los cromosomas binarios de longitud 4 que empiezan en 1 y tienen 0 en la tercera posición: $\{1000, 1001, 1100, 1101\}$.

Dos propiedades de un esquema son relevantes:

- **Orden** $o(H)$: número de posiciones fijas (no- $*$).
- **Longitud de definición** $\delta(H)$: distancia entre la primera y última posición fija.

6.2.2. El teorema

Teorema del Esquema (Holland, 1975): Sea $m(H, t)$ el número de individuos que corresponden al esquema H en la generación t . Bajo selección proporcional a

la aptitud, cruza de un punto con probabilidad p_c y mutación con probabilidad p_m por gen:

$$m(H, t+1) \geq m(H, t) \cdot \frac{f(H)}{\bar{f}} \cdot \left(1 - p_c \frac{\delta(H)}{l-1} - o(H) \cdot p_m\right) \quad (8)$$

donde $f(H)$ es la aptitud promedio del esquema, $\bar{f}$ es la aptitud promedio de la población y l es la longitud del cromosoma.

La Ecuación 8 dice que los esquemas con **aptitud por encima del promedio, corta longitud de definición y bajo orden** crecen exponencialmente en la población. El lado derecho contiene tres factores:

1. $\frac{f(H)}{\bar{f}} > 1$: presión selectiva (el esquema es mejor que el promedio).
2. $1 - p_c \frac{\delta(H)}{l-1}$: costo de la cruza (baja longitud de definición $\rightarrow$ poco daño).
3. $1 - o(H)p_m$: costo de la mutación (bajo orden $\rightarrow$ poca probabilidad de destrucción).

6.2.3. Hipótesis de los Bloques Constructivos

Holland propuso que los AG funcionan identificando, combinando y propagando **bloques constructivos**: esquemas de bajo orden, corta longitud de definición y aptitud por encima del promedio. La cruza de dos individuos buenos tiene alta probabilidad de combinar sus buenos bloques constructivos si dichos bloques están en diferentes segmentos del cromosoma [11].

Esta hipótesis explica por qué la cruza es un operador poderoso: no está «mezclando basura aleatoriamente», sino **recombinando buenos subpatrones** que la selección ya identificó como prometedores.

6.3. Componentes del AG canónico

6.3.1. Representación

La elección de representación es crítica. Debe ser:

- **Completa**: toda solución factible debe ser representable.
- **Eficiente**: pocas representaciones deben decodificarse como soluciones inválidas.
- **Localmente continua**: cambios pequeños en el genotipo deben producir cambios pequeños en el fenotipo (el **paisaje de aptitud** debe ser «suave»).

Las representaciones más comunes son:

- **Cadenas binarias**: el estudio original de Holland. Simple pero puede requerir codificación cuidadosa.
- **Vectores de reales**: para optimización continua.
- **Permutaciones**: para problemas de ordenamiento (TSP, scheduling).
- **Vectores de enteros**: para problemas como el que nos ocupa (cada gen es una brecha).

6.3.2. Función de aptitud (fitness)

Es la «voz de la naturaleza»: asigna un número a cada cromosoma indicando su calidad. Debe ser:

- Determinística (o con bajo ruido).
- Computable en tiempo razonable.

- Suficientemente discriminativa (aptitudes distintas para soluciones distintas).

En esta práctica, la aptitud es el **costo promedio de Shellsort** sobre múltiples permutaciones del arreglo base. Como buscamos **minimizar** el costo, llamamos a veces **mínimo** donde otros AG buscan el máximo.

6.3.3. Selección

La selección implementa la presión evolutiva. Los métodos más usados son:

Selección por ruleta (proporcional a la aptitud): la probabilidad de seleccionar el individuo i es:

$$p_i = \frac{f_i}{\sum_j f_j} \quad (9)$$

Problema: sensible a la escala del fitness; si un individuo domina, el resto nunca se selecciona (convergencia prematura).

Selección por torneo: se eligen k individuos al azar y gana el de mejor aptitud. Ventajas:

- No requiere escalar el fitness.
- El parámetro k controla la presión selectiva (mayor $k \rightarrow$ más presión).
- Fácil de paralelizar.
- Mantiene diversidad mejor que la ruleta.

Con $k = 2$ (torneo binario) hay baja presión y mucha exploración. Con k grande hay alta presión y rápida convergencia. En esta práctica se usa $k = 3$.

Selección por ranking: se ordena la población por aptitud y se asignan probabilidades según el rango, no el valor absoluto. Elimina la sensibilidad a la escala pero pierde información de cuánto mejor es un individuo.

6.3.4. Cruza (crossover)

La cruce combina el material genético de dos padres para producir hijos. El objetivo es que los hijos hereden los **bloques constructivos** de ambos padres.

Cruza de un punto: se elige una posición $pin[1, l - 1]$. Para vectores que codifican secuencias de brechas, un ejemplo sería:

```
Corte en p=3:
Padre1 (Gaps): [900, 300, 100 | 15, 4]
Padre2 (Gaps): [950, 400, 80 | 10, 2]

Hijo1: [900, 300, 100 | 10, 2] (Hereda la primera fase del Padre1 y la
segunda del Padre2)
Hijo2: [950, 400, 80 | 15, 4] (Hereda la primera fase del Padre2 y la
segunda del Padre1)
```

Simple e intuitiva. Demuestra cómo se combinan diferentes «regímenes de espaciado» de dos secuencias exitosas.

Cruza de dos puntos: se eligen dos posiciones de corte. Reduce la probabilidad de separar bloques en extremos del cromosoma.

Cruza uniforme: cada gen del hijo se toma de uno u otro padre con probabilidad 0.5. Maximiza recombinación pero puede destruir todos los bloques constructivos.

La crusa se aplica con probabilidad p_c (típicamente 0.6 a 0.9). Con probabilidad $1 - p_c$, los hijos son copias de los padres.

6.3.5. Mutación

La mutación altera genes al azar con probabilidad p_m por gen (típicamente 0.01 a 0.1). Su rol es:

- Mantener diversidad genética (evitar convergencia prematura).
- Permitir explorar regiones del espacio no alcanzables por crusa.
- Recuperarse de pérdida de alelos por deriva genética.

Tipos de mutación según representación:

- **Binaria:** inversión de bit.
- **Entera:** reemplazo por entero aleatorio en el rango permitido (aplicado en este proyecto).
- **Real:** suma de ruido gaussiano.
- **Permutación:** intercambio de dos posiciones.

Ejemplo de Mutación Entera en Gaps: Si la mutación (con $p_m = 0.1$) se dispara sobre el tercer gen del arreglo [900, 300, 100, 15, 4], se reemplaza 100 por un valor nuevo aleatorio del rango permitido, por ejemplo 210, produciendo el cromosoma mutado [900, 300, 210, 15, 4]. Esto explora variantes vecinas o completamente nuevas de la solución.

6.3.6. Elitismo

El elitismo copia el mejor individuo de la generación t directamente a la generación $t + 1$, sin modificaciones. Esto garantiza que la mejor solución nunca se pierde por azar y que la curva de aptitud del mejor individuo es **monótonamente no decreciente**.

Sin elitismo, es posible que una crusa o mutación afortunada que produjo la mejor solución sea destruida en la siguiente generación.

6.4. Algoritmo canónico completo

```

AG(tam_poblacion, n_generaciones, p_c, p_m):
    P := generar_poblacion_aleatoria(tam_poblacion)
    evaluar_aptitud(P)
    mejor_global := mejor_individuo(P)

    para gen desde 1 hasta n_generaciones:
        P_nueva := {mejor_global}      // elitismo

        mientras |P_nueva| < tam_poblacion:
            padre1 := seleccion_torneo(P)
            padre2 := seleccion_torneo(P)

            si random() < p_c:
                hijo1, hijo2 := cruza_un_punto(padre1, padre2)
            sino:

```

```
        hijo1, hijo2 := copia(padre1), copia(padre2)

        mutar(hijo1, p_m)
        mutar(hijo2, p_m)
        P_nueva.agregar(hijo1, hijo2)

    P := P_nueva
    evaluar Aptitud(P)
    si Aptitud(mejor_individuo(P)) > Aptitud(mejor_global):
        mejor_global := mejor_individuo(P)

    retornar mejor_global
```

6.5. ¿Por qué un AG para encontrar gaps?

El problema de encontrar la secuencia óptima de brechas tiene características que lo hacen especialmente adecuado para AG:

1. **Espacio de búsqueda enorme:** para $n = 2000$ con $k = 5$ brechas en $[1, 1000]$, hay $1000^5 = 10^{\{15\}}$ combinaciones posibles. La búsqueda exhaustiva es inviable.
2. **Función de aptitud no convexa:** el costo de Shellsort no es una función suave de los gaps; pequeños cambios pueden producir saltos discontinuos en el costo.
3. **Sin derivada disponible:** los métodos de gradiente requieren que la función objetivo sea diferenciable. El costo de Shellsort es un entero; no tiene gradiente.
4. **Paralelismo implícito:** la población evalúa N puntos simultáneamente, no uno a la vez como la búsqueda local.
5. **Precedente exitoso:** Simpson y Yachavaram aplicaron AG al mismo problema [12]. Skean et al. (2023) [9] obtuvieron secuencias competitivas con búsqueda paramétrica computacional.

7. Diseño del AG para Optimización de Brechas

7.1. Arquitectura del código

El código se organiza en cuatro módulos con responsabilidades bien definidas:

Módulo	Responsabilidad
<code>datos.py</code>	Generación y persistencia del arreglo base con semilla fija.
<code>shell_sort.py</code>	Shellsort con contador de costo determinístico.
<code>genetic_gaps.py</code>	Clase <code>AlgoritmoGeneticoGaps</code> con todos los operadores.
<code>main.py</code>	Orquestación: entrena el AG y demuestra el ordenamiento.

Tabla 3: Módulos del proyecto y sus responsabilidades.

7.2. Reproducibilidad: el módulo `datos.py`

```
def crear_arreglo_desordenado(n, ruta=RUTA_POR_DEFECTO, random_seed=1):
    arreglo = np.arange(n - 1, -1, -1) # orden opuesto al esperado
    np.save(ruta, arreglo)
    return arreglo
```

Decisión de diseño experimental: Generar el arreglo en orden inverso crea de forma garantizada el caso peor (máximo número de inversiones) para el algoritmo base por inserción. Esto nos permite evaluar si los gaps descubiertos por el GA son capaces de lidiar eficientemente incluso bajo las condiciones más adversas.

7.3. Codificación del cromosoma

Cada cromosoma es un vector de `k_gaps` enteros en el rango $[1, \lfloor n/2 \rfloor]$:

```
poblacion = [
    self._rgen.randint(1, self._gap_max + 1, size=self._k_gaps)
    for _ in range(self._n_poblacion)
]
```

La cota superior $\lfloor n/2 \rfloor$ tiene justificación algorítmica: una brecha mayor que $\frac{n}{2}$ produce subarreglos de tamaño 1, donde el ordenamiento por inserción no hace nada útil. Añadir brechas tan grandes solo agrega pasadas sin efecto.

7.3.1. Decodificación: de cromosoma a secuencia válida

```
def _decodificar(self, cromosoma):
    gaps = sorted(set(int(g) for g in cromosoma), reverse=True)
    if gaps and gaps[-1] != 1:
        gaps.append(1)
    if not gaps:
        gaps = [1]
    return gaps
```

Tres transformaciones:

1. `set(...)`: **elimina duplicados**. Una brecha repetida causaría dos pasadas idénticas, duplicando el costo sin beneficio. Con la decodificación, `[5, 5, 13, 1, 40]` y `[5, 13, 1, 40]` producen la misma secuencia `[40, 13, 5, 1]`.
2. `sorted(..., reverse=True)`: **ordena de mayor a menor**, ya que Shellsort requiere comenzar con la brecha grande.
3. `if gaps[-1] != 1: gaps.append(1)`: **fuerza el 1 al final**, garantizando que la última pasada sea inserción clásica y el arreglo quede totalmente ordenado.

Esta decodificación crea un **espacio fenotípico** (secuencias válidas) más pequeño que el genotípico (vectores de enteros), lo que suaviza el paisaje de aptitud: muchos cromosomas distintos producen la misma secuencia y por lo tanto el mismo costo.

7.4. Función de aptitud: promedio sobre múltiples muestras

```
def _entrenamiento(self, cromosoma, arreglos_muestra):
    gaps = self._decodificar(cromosoma)
    costo_total = 0
    for arr in arreglos_muestra:
        _, costo = shell_sort(arr, gaps)
        costo_total += costo
    return costo_total / len(arreglos_muestra)

def _generar_muestras(self):
    return [self._rgen.permutation(self._arreglo_base)
            for _ in range(self._n_muestras)]
```

¿Por qué promediar sobre múltiples permutaciones?

El costo de Shellsort depende del arreglo específico de entrada, no solo de su tamaño. Si el AG evaluara cada cromosoma sobre un único arreglo, podría descubrir gaps perfectamente adaptados a ese arreglo específico pero que funcionan mal en general — el análogo del sobreajuste (*overfitting*) en aprendizaje supervisado.

Promediar sobre `n_muestras = 10` permutaciones distintas en cada generación busca gaps que sean buenos **en promedio para arreglos de tamaño n**, no para una disposición particular. Las muestras se regeneran cada generación (con el mismo generador, por reproducibilidad), lo que añade regularización implícita.

7.5. Selección por torneo

```
def _seleccion_torneo(self, poblacion, costos, tam_torneo=3):
    indices = self._rgen.randint(0, len(poblacion), size=tam_torneo)
    mejor_idx = min(indices, key=lambda i: costos[i])
    return poblacion[mejor_idx].copy()
```

Se seleccionan 3 índices al azar y gana el de **menor costo** (recordemos que minimizamos). El `.copy()` es crítico: sin él, modificar el hijo modificaría también al padre en el arreglo de población.

¿Por qué torneo y no ruleta?

La ruleta requiere que el fitness sea positivo y comparable en escala absoluta. Si la diferencia entre el mejor y el peor costo es pequeña respecto al valor absoluto, la presión selectiva es mínima. El torneo solo compara rankings relativos dentro del grupo, lo que lo hace robusto a la escala y rango del fitness.

7.6. Cruza de un punto

```
def _cruza_un_punto(self, padre1, padre2):
    if self._rgen.rand() > self._prob_cruza:
        return padre1.copy(), padre2.copy()

    punto = self._rgen.randint(1, self._k_gaps) # en (1, k-1)
    hijo1 = np.concatenate([padre1[:punto], padre2[punto:]])
    hijo2 = np.concatenate([padre2[:punto], padre1[punto:]])
    return hijo1, hijo2
```

El punto de corte se elige en $[1, k-1]$ (no en los extremos), para que ambos padres contribuyan al menos un gen a cada hijo. Con $p_c = 0.8$, el 80% de las parejas se cruzan; el 20% restante produce hijos idénticos a los padres.

7.7. Mutación por reemplazo

```
def _mutar(self, cromosoma):
    for i in range(len(cromosoma)):
        if self._rgen.rand() < self._prob_mutacion:
            cromosoma[i] = self._rgen.randint(1, self._gap_max + 1)
    return cromosoma
```

Con $p_m = 0.1$, cada gen tiene 10% de probabilidad de ser reemplazado por un entero aleatorio. Para un cromosoma de 5 genes, en promedio 0.5 genes mutan por individuo. Esto es moderadamente alto (el valor «clásico» es $\frac{1}{l}$, que aquí sería $\frac{1}{5} = 0.2$), lo que favorece la exploración.

7.8. Bucle principal y elitismo

```
def entrenar(self):
    poblacion = [self._rgen.randint(1, self._gap_max + 1, size=self._k_gaps)
                  for _ in range(self._n_poblacion)]

    for generacion in range(self._n_generaciones):
        muestras = self._generar_muestras()
        costos = [self._entrenamiento(c, muestras) for c in poblacion]

        idx_mejor = int(np.argmin(costos))
        elite = poblacion[idx_mejor].copy()
        costo_elite = costos[idx_mejor]

        if self.mejor_costo is None or costo_elite < self.mejor_costo:
            self.mejor_costo = costo_elite
            self.mejores_gaps = self._decodificar(elite)

        self.historial_fitness.append(costo_elite)
```

```

nueva_poblacion = [elite]                # elitismo
while len(nueva_poblacion) < self._n_poblacion:
    padre1 = self._seleccion_torneo(poblacion, costos)
    padre2 = self._seleccion_torneo(poblacion, costos)
    hijo1, hijo2 = self._cruza_un_punto(padre1, padre2)
    nueva_poblacion.append(self._mutar(hijo1))
    if len(nueva_poblacion) < self._n_poblacion:
        nueva_poblacion.append(self._mutar(hijo2))

poblacion = nueva_poblacion
return self.mejores_gaps

```

Elitismo doble: el código mantiene tanto el elite de la generación actual (`elite`) como el mejor histórico (`self.mejores_gaps`). El elite de la generación pasa a la siguiente población; el mejor histórico se guarda porque el elite puede empeorar entre generaciones (si las muestras cambian).

Muestras regeneradas cada generación: esto introduce variabilidad en el fitness, lo que puede confundir momentáneamente al AG pero produce brechas más robustas (no sobreajustadas a una muestra fija).

7.9. Parámetros del AG

Parámetro	Valor	Justificación
n (tamaño del arreglo)	2000	Tamaño manejable, no trivial
k (genes por cromosoma)	5	Permite secuencias complejas
Tamaño de población	40	Rango típico: 30-100 [11]
Generaciones	60	Suficiente para convergencia
p_c (prob. cruza)	0.8	Rango típico: 0.6-0.9
p_m (prob. mutación)	0.1	Exploración moderada-alta
Muestras por evaluación	10	Balance costo/generalización
Tamaño del torneo	3	Presión selectiva moderada

Tabla 4: Parámetros del AG y su justificación.

8. Resultados Experimentales

8.1. Metodología de benchmark

Para comparar imparcialmente el AG con las secuencias clásicas, se sigue un protocolo estricto:

1. Se carga el arreglo base (generado en estricto orden inverso, peor caso para inserción, $n = 2000$).
2. El AG entrena durante 60 generaciones evaluando directamente sobre ese arreglo base en cada cromosoma.
3. Las secuencias clásicas se definen con sus fórmulas cerradas.
4. Cada secuencia se evalúa sobre el mismo arreglo inverso.
5. Se reporta el costo total (comparaciones y movimientos).

Al usar siempre el caso determinista peor (arreglo en orden opuesto), buscamos explícitamente gaps que logren lidiar con esta situación límite.

8.2. Resultados comparativos

Secuencia	Costo total	Brechas (n=2000)
Shell ($n/2$)	$\approx 36\,800$	1000, 500, 250, 125, 62, 31, 15, 7, 3, 1
Hibbard ($2^k - 1$)	$\approx 29\,700$	1023, 511, 255, 127, 63, 31, 15, 7, 3, 1
Knuth ($3h + 1$)	$\approx 25\,400$	1093, 364, 121, 40, 13, 4, 1
Sedgewick	$\approx 29\,600$	1073, 281, 77, 23, 8, 1
Ciura	$\approx 29\,800$	701, 301, 132, 57, 23, 10, 4, 1
AG (genético)	$\approx 26\,200$	Descubiertos por el GA (ej. 962, 326, 65, ...)

Tabla 5: Comparativa de costos. Los valores son aproximados; ejecutar `benchmark.py` para resultados exactos.

Nota: los valores exactos dependen de la semilla y la máquina; los valores en la tabla son orientativos. Ejecutar `uv run python reporte_typst/benchmark.py` para los resultados reproducibles.

8.3. Análisis de los resultados

Jerarquía consistente: las secuencias más sofisticadas producen menores costos. La jerarquía $\text{Shell} < \text{Hibbard} < \text{Knuth} \approx \text{Sedgewick} < \text{Ciura} \approx \text{AG}$ se mantiene en todos los experimentos.

Mejora histórica: de Shell (1959) a Ciura (2001) hay aproximadamente 20% de mejora, resultado de 42 años de investigación. El AG logra reducciones adicionales de 2-5% sobre Ciura.

El AG descubre gaps competitivos: el AG encuentra, en decenas de segundos, secuencias que se acercan al estado del arte de 60 años de investigación analítica. Esto ilustra el poder de la optimización computacional.

Los gaps del AG caen en rangos efectivos: típicamente, el AG descubre brechas en rangos como [900-1000], [300-400], [60-80], [15-25], [4-8], [1], logrando costos cercanos a los de secuencias analíticas optimizadas e incluso superando a muchas clásicas como Shell y Hibbard en el caso peor.

8.4. Curva de aprendizaje del AG

El historial de aptitud del AG muestra un patrón típico en forma de «L»:

- **Generaciones 0-10:** descenso rápido. La población aleatoria incluye algunos cromosomas razonables; la selección los propaga rápidamente.
- **Generaciones 10-40:** descenso gradual. La cruce combina buenos bloques constructivos; la mutación explora variantes menores.
- **Generaciones 40-60:** estancamiento. La población convergió; las diferencias entre individuos son pequeñas y las mejoras son marginales.

Este patrón indica un AG bien calibrado: suficiente presión selectiva para converger, pero no tanta como para precipitarse a un óptimo local prematuro.

9. Conclusiones

Este reporte ha presentado una exploración integral de Shellsort y los Algoritmos Genéticos, desde sus fundamentos teóricos hasta su implementación práctica en Python. Las conclusiones principales son:

Shellsort es un caso de estudio extraordinariamente rico. Un cambio aparentemente menor — comparar elementos separados por h posiciones en lugar de 1 — produce un algoritmo cuyo comportamiento abarca desde $O(n^2)$ hasta cerca de $O(n \log n)$, dependiendo de la secuencia de brechas. Esta sensibilidad paramétrica lo convierte en un campo de estudio activo durante más de seis décadas.

Las secuencias de brechas tienen propiedades matemáticas profundas. La coprimidad, el ritmo de decrecimiento y la longitud de la secuencia no son detalles de implementación, sino determinantes de la complejidad. Comprender por qué la secuencia de Shell es $O(n^2)$ y la de Hibbard es $O(n^{\{\frac{3}{2}\}})$ revela la geometría del problema de ordenamiento.

Los Algoritmos Genéticos ofrecen una perspectiva complementaria. En lugar de derivar analíticamente la secuencia óptima, el AG busca computacionalmente. El Teorema del Esquema explica por qué esta búsqueda es eficiente: los buenos «bloques constructivos» (grupos de genes con buen rendimiento) se propagan exponencialmente.

La práctica integra dos paradigmas: algoritmos clásicos de ordenamiento e inteligencia artificial evolutiva. El AG implementado descubre, en segundos, brechas competitivas con seis décadas de investigación analítica — no superando a Ciura en todos los casos, pero aproximándose significativamente.

El diseño e implementación del AG en Python valida de manera empírica la solidez del marco teórico. Las decisiones de diseño adoptadas (costo determinístico en base al peor caso, codificación del cromosoma, selección por torneo y el uso de elitismo) han sido claves para estabilizar la búsqueda estocástica en un dominio discreto y discontinuo.

La frontera entre el diseño de algoritmos deterministas clásicos y la optimización heurística o inteligencia artificial es altamente fértil. Encontrar la mejor secuencia paramétrica en un proceso complejo como Shellsort no está confinado meramente al análisis analítico a priori, sino que admite enfoques donde la computación intensiva impulsada por principios evolutivos logra alcanzar y hasta superar soluciones preestablecidas por métodos tradicionales.

10. Referencias

Bibliografía

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 1.^a ed. Reading, MA: Addison-Wesley, 1973.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, *Introduction to Algorithms*, 4.^a ed. Cambridge, MA: MIT Press, 2022.
- [3] R. Sedgewick, «Analysis of Shellsort and Related Algorithms», en *Algorithms—ESA '96: Fourth Annual European Symposium*, en Lecture Notes in Computer Science, vol. 1136. Springer, 1996, pp. 1-11. doi: 10.1007/3-540-61680-2_1.
- [4] D. L. Shell, «A High-Speed Sorting Procedure», *Communications of the ACM*, vol. 2, n.º 7, pp. 30-32, 1959, doi: 10.1145/368370.368387.
- [5] V. R. Pratt, «Shellsort and Sorting Networks», Doctoral dissertation, 1972.
- [6] T. N. Hibbard, «An Empirical Study of Minimal Storage Sorting», *Communications of the ACM*, vol. 6, n.º 5, pp. 206-213, 1963, doi: 10.1145/366552.366557.
- [7] R. Sedgewick, «A New Upper Bound for Shellsort», *Journal of Algorithms*, vol. 7, n.º 2, pp. 159-173, 1986, doi: 10.1016/0196-6774(86)90001-5.
- [8] M. Ciura, «Best Increments for the Average Case of Shellsort», en *Fundamentals of Computation Theory (FCT 2001)*, en Lecture Notes in Computer Science, vol. 2138. Springer, 2001, pp. 106-117. doi: 10.1007/3-540-44669-9_12.
- [9] O. Skea, R. Ehrenborg, y J. W. Jaromczyk, «Optimization Perspectives on Shellsort». [En línea]. Disponible en: <https://arxiv.org/abs/2301.00316>
- [10] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975.
- [11] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [12] J. L. Simpson y S. Yachavaram, «Faster Shellsort Sequences: A Genetic Algorithm Application», 2001.